

RSA Factorization Attacks and Private-Key Recovery

Mateo Oñate

School of Mathematical and Computational Sciences, Yachay Tech
Email: mateo.onate@yachaytech.edu.ec

ABSTRACT We implement Trial Division, Fermat factorization, and Pollard’s Rho to recover the private key of a small Rivest–Shamir–Adleman cryptosystem. Each algorithm factors the public modulus independently. We then calculate Euler’s totient, obtain the private exponent, decrypt the ciphertext, and check the result through encryption. All three methods recover the factors 53 and 61 from the required modulus 3233 and return the message 65 from ciphertext 2790. Our execution passes 21 automated tests before the timing experiments. We compare four generated moduli from 16 to 28 bits, with three measurements per algorithm and modulus on the same Windows computer. For the largest modulus, mean times are 0.928667 milliseconds for Trial Division, 0.001500 milliseconds for Fermat, and 0.133867 milliseconds for Pollard’s Rho. The two factors of this case differ by 132, which favors Fermat. A separate comparison between two 28-bit moduli changes the prime distance from 10 to 45018. Fermat then requires 1 and 12419 checks, respectively. These results connect factor structure with execution cost and show how recovering the factors permits private-key reconstruction.

INDEX TERMS Fermat factorization, integer factorization, Pollard’s Rho, RSA cryptanalysis, trial division.

I. INTRODUCTION

RSA represents messages as integers and separates the public exponent from the private exponent. The public modulus is the product of two distinct primes. Factoring this product supplies the information needed to reconstruct the private key, as presented in the course lecture by Iza (2026).

Our objective is to implement the three attacks required by the laboratory handout, verify private-key recovery, and compare their behavior on small generated moduli. We also examine how the distance between the primes affects Fermat. The measurements in this report come from our computer’s exported `resultados.json` file, dated September 23, 2026.

II. FACTORIZATION ALGORITHMS

A. Trial Division

We first test whether 2 divides n . For odd inputs, we check odd candidates from 3 through $\lfloor \sqrt{n} \rfloor$. When a candidate p divides n exactly, we calculate $q = n/p$. At least one factor of a composite integer does not exceed its square root. The implementation counts each divisor check, including the initial check of 2.

B. Fermat Factorization

Fermat searches for a difference of squares:

$$n = a^2 - b^2 = (a - b)(a + b). \quad (1)$$

We start at $a = \lceil \sqrt{n} \rceil$ and calculate $b^2 = a^2 - n$. An exact integer square root determines whether this difference is a perfect square. If it is, we return $a - b$ and $a + b$. Otherwise, we increase a by one. We handle even inputs directly through the factor 2 and reject the trivial factor pair 1 and n .

C. Pollard’s Rho

We generate a modular sequence with

$$f(x) = (x^2 + k) \bmod n. \quad (2)$$

The constant k is separate from the RSA ciphertext c . One position advances once per iteration, and the other advances twice. We then calculate

$$g = \gcd(|x - y|, n). \quad (3)$$

If $1 < g < n$, we have a nontrivial factor. A value of 1 continues the search. When $g = n$, we restart with a different constant and initial position. The benchmark starts each call with $k = 1$ and $x = y = 2$. A factor can emerge when the positions agree modulo one prime but not the other.

III. PRIVATE-KEY RECOVERY

The handout specifies $n = 3233$, $e = 17$, and $c = 2790$. Each method recovers $p = 53$ and $q = 61$. Their order does not affect the RSA calculations. We obtain

$$\phi(n) = (p - 1)(q - 1) = 3120, \quad (4)$$

$$d = e^{-1} \bmod \phi(n) = 2753. \quad (5)$$

Euclid’s extended algorithm supplies the modular inverse. The result satisfies $17 \cdot 2753 \bmod 3120 = 1$. The reconstructed private key is $(3233, 2753)$, and decryption returns

$$m = c^d \bmod n = 65. \quad (6)$$

We encrypt the recovered message again and obtain $65^{17} \bmod 3233 = 2790$. The exported verification value is true for all three methods.

TABLE 1. Recovery of the required RSA example.

Method	p	q	Checks	Restarts
Trial Division	53	61	27	0
Fermat	53	61	1	0
Pollard's Rho	53	61	6	0

TABLE 2. Generated benchmark moduli and prime distances.

Bits	n	p	q	$ p - q $
16	47083	197	239	42
20	732349	739	991	252
24	12277201	3067	4003	936
28	256796269	15959	16091	132

Table 1 reports the work counters. Fermat finds $57^2 - 4^2 = 3233$ on its first check. Rho reaches $x = 968$ and $y = 279$ on iteration 6. The greatest common divisor of their difference, 689, and 3233 is 53. This calculation demonstrates the role of the greatest common divisor in the attack. The three counters describe different operations, so they do not represent equal amounts of computation.

IV. IMPLEMENTATION AND VALIDATION

Our project contains five Python files. The main script reads n , e , and c from the keyboard and runs the seven exercises in order. Separate files contain factorization, RSA recovery, automated tests, and experiments. The attack functions receive only n . The experimental generator retains p and q for checks after timing, rather than supplying them to an attack.

We implement the factorization loops, Euclid's algorithm, the modular inverse, and Square-and-Multiply directly. Python's standard library supplies the exact integer square root, timing, and test framework. No external factoring library participates in the calculation. The RSA routines require distinct prime factors and a public exponent coprime with the totient.

Our result file records 21 approved tests before the experimental evaluation. They cover the greatest common divisor, modular inverses, integer square roots, all three attacks, factor products, and RSA encryption and decryption. Other checks address invalid inputs, generated bit lengths, and restart behavior. For the restart test, $n = 91$, $k = 0$, and initial $x = 0$ force $g = n$. The algorithm then changes its parameters and finds 7 and 13.

V. EXPERIMENTAL METHOD

We ran the experiments on the same computer with Python 3.12.7 and Windows 10, build 19045. The export identifies the processor as Intel64 Family 6, Model 126, Stepping 5, GenuineIntel. It does not record a commercial processor model, clock frequency, or memory capacity.

For the benchmark, we generated one modulus at each of four sizes: 16, 20, 24, and 28 bits. The generator starts with seed $2026 + b$ for a requested size b . It requires distinct primes and checks the actual product bit length. Table 2 lists the resulting cases with factors in ascending order.

For each modulus, we run Trial Division, Fermat, and Rho in this order. Each method receives one untimed warm-up and then three timed calls. We measure the factorization call with

TABLE 3. Mean factorization time in milliseconds. Parentheses contain sample standard deviations.

Bits	n	Trial Division	Fermat	Pollard's Rho
16	47083	0.008933 (0.000321)	0.001433 (0.000252)	0.005733 (0.000681)
20	732349	0.024167 (0.003553)	0.002733 (0.000153)	0.013467 (0.000115)
24	12277201	0.133300 (0.033100)	0.013167 (0.004100)	0.017267 (0.000252)
28	256796269	0.928667 (0.358433)	0.001500 (< 0.000001)	0.133867 (0.000666)

`perf_counter`. Generation, external factor checks, terminal output, and file writing remain outside the timed interval. This produces 36 benchmark measurements. The distance experiment adds six measurements.

We retain every recorded time and calculate the arithmetic mean and sample standard deviation:

$$\bar{t} = \frac{t_1 + t_2 + t_3}{3}, \quad s = \sqrt{\frac{\sum_{i=1}^3 (t_i - \bar{t})^2}{2}}. \quad (7)$$

The report converts seconds from the export to milliseconds. Repeated Rho calls follow the same parameters for a given modulus, so this experiment measures timing variation rather than variation across random starting points.

VI. BENCHMARK RESULTS

Table 3 and Fig. 1 present the measured times. Fermat has the lowest mean for all four selected moduli. The largest case requires one Fermat check because its primes differ by only 132. Its mean is 0.001500 ms, compared with 0.133867 ms for Rho and 0.928667 ms for Trial Division. This answers the comparison for the largest tested modulus without implying the same ranking for every prime pair.

Trial Division increases from 99 divisor checks at 16 bits to 7980 at 28 bits. Its mean time grows by about 104 times across these cases. When the smaller prime increases, Trial Division checks more candidates. Bit length alone does not fix the cost, since an early small divisor would end the search sooner.

The counters also clarify the nonmonotonic Fermat result. Its checks are 2, 10, 32, and 1 as the modulus size increases. The corresponding Rho counts are 6, 14, 9, and 114. Factor structure and sequence behavior matter alongside input size.

The 28-bit Trial Division times range from 0.541700 to 1.249300 ms, with a sample standard deviation of 0.358433 ms. We retain this spread rather than selecting the fastest repetition. Three measurements and one modulus per size provide a limited comparison. Fixed execution order and ordinary system activity also limit timing interpretation. Nearly identical microsecond measurements, such as the 28-bit Fermat results, do not establish precise hardware-level timing stability.

VII. PRIME DISTANCE AND FERMAT

We generated two 28-bit moduli near the same magnitude. The close pair is 14197 and 14207, with product 201696779. The separated pair is 4099 and 49117, with product 201330583. Table 4 and Fig. 2 summarize the comparison.

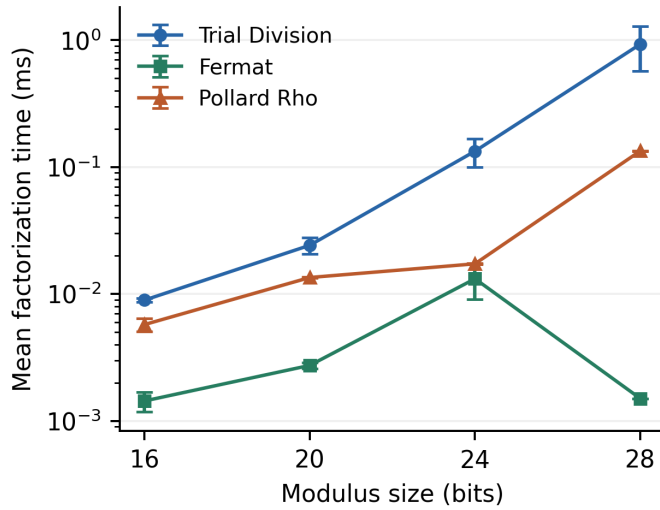


FIGURE 1. Mean factorization times for the four moduli. The vertical axis is logarithmic. Error bars represent one sample standard deviation.

TABLE 4. Fermat results for two 28-bit moduli. Times are in milliseconds.

Quantity	Close primes	Separated primes
$ p - q $	10	45018
Checks	1	12419
Mean time	0.002067	3.054167
Sample standard deviation	0.000351	1.570763

For odd prime factors, the successful Fermat value is $a = (p + q)/2$. Counting the initial candidate gives

$$I = \frac{p+q}{2} - \lceil \sqrt{pq} \rceil + 1. \quad (8)$$

The close case starts and finishes at $a = 14202$. The separated case starts at 14190 and finishes at 26608, giving $26608 - 14190 + 1 = 12419$ checks. Close primes reduce the gap between the initial and final values of a .

The mean time rises from 0.002067 to 3.054167 ms, about 1478 times. The separated case records 4.867300, 2.106300, and 2.188900 ms. We retain all three observations. Their spread prevents attributing the full time ratio solely to arithmetic work, but the exact iteration counts confirm the effect of prime distance.

VIII. SECURITY DISCUSSION

The recovery experiment shows why the factors matter. Once we obtain p and q , the totient follows directly and the modular inverse produces d . Our recovered private exponent decrypts 2790 to 65, and encryption returns the original ciphertext.

These measurements concern small educational moduli. They do not estimate the cost of factoring production-size RSA keys. The deterministic seed supports repeatable experiments, not secure key generation. We implement integer RSA operations without randomized padding. The course lecture distinguishes these operations from padded RSA and hybrid encryption, where RSA protects a short value rather than a complete data file (Iza, 2026).

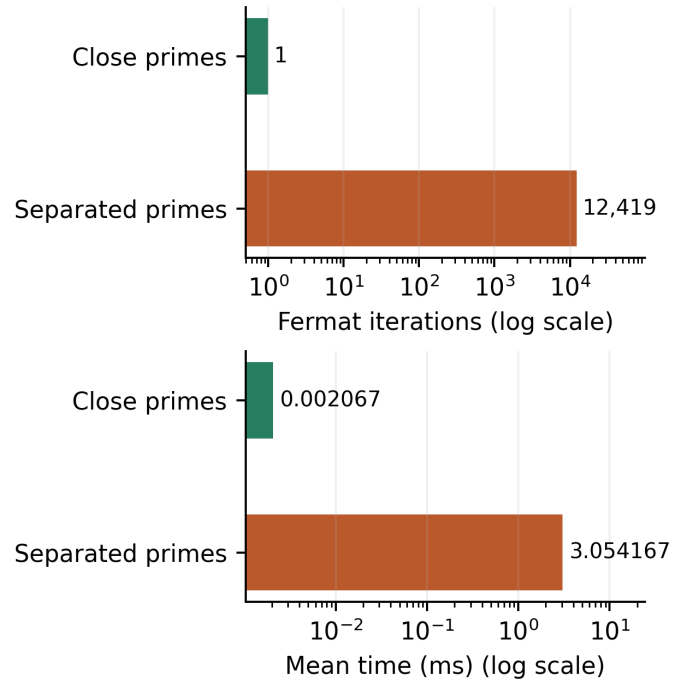


FIGURE 2. Fermat checks and mean execution time for close and separated primes. Both horizontal axes are logarithmic.

IX. CONCLUSIONS

All three implementations recovered the required factors, private exponent, and message. Our execution passed 21 tests before measurement. Trial Division performed more checks as the smaller factors grew. Fermat recorded the shortest mean for the largest selected modulus because its factors were close. At the same 28-bit size, separating the primes increased Fermat's work from 1 to 12419 checks. The measured variation supports reporting the raw observations and their spread alongside the averages.

REFERENCES

- Cryptography course. *Lab 4: RSA Factorization Attacks*. Yachay Tech University, course handout, pp. 1–5.
- Iza, C. *Public-Key Cryptography – RSA*. Lecture 6, Yachay Tech University, September 20, 2026, slides 16–43, 49–56, and 61–74.